```
from google.colab import drive
drive.mount('/content/drive')


```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
# 2. Add the directory containing 'models.py' to Python's path
import sys
sys.path.append('/content/drive/MyDrive/') # Assuming 'models.py' is directly in MyDrive

# 3. Import the module
import models
from models import *
```

```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_quant_8x8_soft_shrink2_128"
model = VGG16_quant_8x8_soft_shrink()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243, 0.262])


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2)


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2)


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
```

```
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))


def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out the status. e.g., i%5 => every 5 batch
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


```

```
def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120 epochs"""
    adjust_list = [300,400]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
=> Building model...
VGG_quant(
  (features): Sequential(
    (0): QuantConv2d(
      3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (1): Softshrink(0.5)
    (2): QuantConv2d(
      64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (3): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): QuantConv2d(
      64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (7): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (8): ReLU(inplace=True)
    (9): QuantConv2d(
      128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (10): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (13): QuantConv2d(
      128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
```

```
    )
    (14): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (15): ReLU(inplace=True)
    (16): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (17): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (18): ReLU(inplace=True)
    (19): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (20): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (21): ReLU(inplace=True)
    (22): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (23): QuantConv2d(
      256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (24): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (25): ReLU(inplace=True)
    (26): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (27): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```
1
2 model.features[0] ## Modified layer
```

```
QuantConv2d(
  3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

```
1 # This cell is from the website
2
3 lr = 8e-3
4
5
6
7 weight_decay = 1e-4
8 epochs = 500
9 best_prec = 0
10
11 model = model.cuda()
12 criterion = nn.CrossEntropyLoss().cuda()
13 optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum = 0.90, weight_decay=weight_decay) #changed momentum ar
14 # weight decay: for regularization to prevent overfitting
15
```

```
1
2
3 if not os.path.exists('result'):
4     os.makedirs('result')
5
6 fdir = 'result/'+str(model_name)
7
8 if not os.path.exists(fdir):
9     os.makedirs(fdir)
10
11
```

```
1
2 for epoch in range(0, epochs):
3     adjust_learning_rate(optimizer, epoch)
4
5     train(trainloader, model, criterion, optimizer, epoch)
6
7     # evaluate on test set
8     print("Validation starts")
9     prec = validate(testloader, model, criterion)
10
11     # remember best precision and save checkpoint
12     is_best = prec > best_prec
13     best_prec = max(prec,best_prec)
14     print('best acc: {:1f}'.format(best_prec))
15     if is_best:
16       save_checkpoint({
17         'epoch': epoch + 1,
18         'state_dict': model.state_dict(),
19         'best_prec': best_prec,
```

```
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)

    if prec > 90: break

```

```
!cp -r  /content/result/VGG16_quant_8x8_soft_shrink2_128/ /content/drive/MyDrive/result

```

```
PATH = '/content/drive/MyDrive/result/VGG16_quant_8x8_soft_shrink2_128/model_best.pth.tar'
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 8654/10000 (87%)
```

```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

######### Save inputs from selected layer ##########
save_output = SaveOutput()
i = 0
count = 0
for layer in model.modules():
    i = i+1
    if isinstance(layer, QuantConv2d):
        count += 1
        print(i,"-th layer prehooked \n", layer, count) #'count' here is just for me to check the index of save_output.
        layer.register_forward_pre_hook(save_output)
####################################################

dataiter = iter(testloader)
images, labels = next(dataiter)
images = images.to(device)
images_abs = torch.abs(images) # taking absolute value of image input since our feature map activations are assumed to
out = model(images_abs)
```

```
 64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 2
11 -th layer prehooked
QuantConv2d(
 64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 3
15 -th layer prehooked
QuantConv2d(
 128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
```

```
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 6
28 -th layer prehooked
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 7
33 -th layer prehooked
 QuantConv2d(
  256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 8
37 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 9
41 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 10
46 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 11
50 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 12
54 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 13
```

```
def int_softshrink(y_int: torch.Tensor,
                   delta_out: float,
                   lambda_real: float) -> torch.Tensor:
    """
    Apply softshrink to an integer-quantized tensor before dequantization.

    y_real = delta_out * y_int
    softshrink(y_real; lambda_real) ≈ delta_out * y_int_shrunk
    """
    # Integer threshold
    lambda_int = torch.round(torch.tensor(lambda_real / delta_out)).to(y_int.dtype)

    # Masks in integer domain
    mask_pos = y_int > lambda_int
    mask_neg = y_int < -lambda_int

    y_shrunk_int = torch.zeros_like(y_int)

    # y - lambda for positive
    y_shrunk_int[mask_pos] = y_int[mask_pos] - lambda_int

    # y + lambda for negative
    y_shrunk_int[mask_neg] = y_int[mask_neg] + lambda_int

    return y_shrunk_int
```

```
weight_q = model.features[0].weight_q
w_alpha = model.features[0].weight_quant.wgt_alpha
w_bit = 4

weight_int = weight_q / (w_alpha / (2**(w_bit-1)-1))
#print(weight_int)
```

```
act = images_abs
act_alpha  = model.features[0].act_alpha
act_bit = 2
act_quant_fn = act_quantization(act_bit)

act_q = act_quant_fn(act, act_alpha)

act_int = act_q / (act_alpha / (2**act_bit-1))
print(torch.all(act_int >= 0)) ## all are positive
```

```
tensor(True, device='cuda:0')
```

```
conv_int = torch.nn.Conv2d(in_channels = 3, out_channels=64, kernel_size = 3, padding=1)
conv_int.weight = torch.nn.parameter.Parameter(weight_int)

conv_int.bias = model.features[0].bias
output_int = conv_int(act_int) # The most important thing
delta_out = (act_alpha / (2**act_bit-1)) * (w_alpha / (2**(w_bit-1)-1))
output_recovered_soft_shrink = int_softshrink(output_int, delta_out, lambda_real = 0.5)
output_recovered = output_recovered_soft_shrink * delta_out

#print(output_recovered)
```

```
/tmp/ipython-input-2470372827.py:11: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.det
  lambda_int = torch.round(torch.tensor(lambda_real / delta_out)).to(y_int.dtype)
```

As you can see from the below result, finding the lambda threshold in the quantized domain, applying it to the output feature map before dequantization, then applying dequantization introduces a very negligable error. This means we can apply the softshrink function as an activation function in the sfp accumulation and activation module.

```
print(save_output.outputs[1][0].shape)
difference = abs(save_output.outputs[1][0] - output_recovered )
print(difference.mean())  ## It should be small, e.g.,2.3 in my trainned model
```

```
torch.Size([128, 64, 32, 32])
tensor(0.0968, device='cuda:0', grad_fn=<MeanBackward0>)
```

The below cell implements the required reshaping of the input activation map and the weight tensors to be used when generating txt files for testing the output stationary case

```
import torch
import torch.nn.functional as F


stride = 1        # or (1, 1)
padding = 1       # use 0 if you had no padding
dilation = 1
groups = 1

use_bias = False  # set True and define bias_int if your conv has bias
bias_int = None   # e.g. torch.randint(..., size=(64,), dtype=torch.int16)

# If you do integer scaling (e.g., >> shift_bits after accumulation)
apply_shift = False
shift_bits = 3    # example: right shift by 3


print("act_int.shape    :", act_int.shape)     # [128, 3, 32, 32]
print("weight_int.shape :", weight_int.shape)  # [64, 3, 3, 3]
print("output_int.shape :", output_int.shape)  # [128, 64, 32, 32]

batch = 5
out_c = 1
h = 0
w = 0


act_f = act_int.to(torch.float32)
weight_f = weight_int.to(torch.float32)

bias_f = None
if use_bias and bias_int is not None:
    bias_f = bias_int.to(torch.float32)

out_f = F.conv2d(
    act_f, weight_f,
    bias=bias_f,
    stride=stride,
    padding=padding,
    dilation=dilation,
    groups=groups,
)

print("F.conv2d reference (float):", out_f[batch, out_c, h, w].item())
print("Stored output_int          :", output_int[batch, out_c, h, w].item())


```

```
# We need to mimic the same padding as conv2d.
# F.pad pads in the order (left, right, top, bottom).
if isinstance(padding, int):
    pad_left = pad_right = pad_top = pad_bottom = padding
else:
    # if you used tuple padding, adjust this section
    pad_left = pad_right = padding[1] if isinstance(padding, tuple) else padding
    pad_top  = pad_bottom = padding[0] if isinstance(padding, tuple) else padding

act_padded = F.pad(act_int, (pad_left, pad_right, pad_top, pad_bottom))

# Kernel size
_, in_channels, kH, kW = weight_int.shape

# For stride=1, dilation=1, the input top-left index for output (h, w) is:
h_in = h * stride
w_in = w * stride

# Extract 3x3 (or kH x kW) patch from padded input
patch_int = act_padded[batch, :, h_in:h_in + kH, w_in:w_in + kW]   # [Cin, kH, kW]
kernel_int = weight_int[out_c]                                     # [Cin, kH, kW]

# Integer accumulation in int32 to avoid overflow
acc_int32 = (patch_int.to(torch.int32) * kernel_int.to(torch.int32)).sum()

if use_bias and bias_int is not None:
    acc_int32 = acc_int32 + int(bias_int[out_c].item())

print("Manual integer accumulator :", acc_int32.item())


if apply_shift:
    # arithmetic right shift for signed values
    acc_shifted = acc_int32 >> shift_bits
else:
    acc_shifted = acc_int32

# Clamp to int16 range
acc_int16 = torch.clamp(acc_shifted, -32768, 32767).to(torch.int16)

print("Manual quantized int16     :", acc_int16.item())
print("Stored output_int          :", output_int[batch, out_c, h, w].item())
```

```
act_int.shape    : torch.Size([128, 3, 32, 32])
weight_int.shape : torch.Size([64, 3, 3, 3])
output_int.shape : torch.Size([128, 64, 32, 32])
F.conv2d reference (float): 16.0
Stored output_int          : 16.0
Manual integer accumulator : 16
Manual quantized int16     : 16
Stored output_int          : 16.0
```

```
batch = 5
X = act_padded[batch,:,:,:]  # pick only one input C_in, ni, nj
w_int = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))  # out_ch = 64, in_ch = 3, kij = 9
W = w_int
out_int = output_int[batch]
out_int = torch.reshape(out_int, (out_int.size(0), -1))

```

```
print(X.shape)
print(W.shape)
```

```
torch.Size([3, 34, 34])
torch.Size([64, 3, 9])
```

```
batch = 5
in_channels = 3
bit_precision = 4
h = 3
k_size = 3
array_size = 8
kij = 9

for i in range(in_channels):
  fname = f"activation_in_ch{i}.txt"
  file = open(fname, 'w') #write to file
  for j in range(h):
    for k in range(h):
      for l in range(array_size):
        #print(round(X[i,j,7-l+k].item()))
```

```
        X_bin = '{0:04b}'.format(round(X[i,j,7-l+k].item()))
        for m in range(bit_precision):
              file.write(X_bin[m])
      file.write('\n')

  file.close()

w_check = []

for i in range(in_channels):
  fname = f"weight_in_ch{i}.txt"
  file = open(fname, 'w') #write to file
  for ker in range(kij):
    for j in range(array_size):
      if (7-j == 2):
        w_check.append(W[7-j,i,ker].item())
        #print(f"o_channel: {7-j}, in_channel {i}, kernel {ker}, value {w_check[-1]}")
      if (round(W[7-j,i,k].item()) >= 0):
          W_bin = '{0:04b}'.format(round(W[7-j,i,ker].item()))
      else:
          W_bin = '{0:04b}'.format(round(W[7-j,i,ker].item())+16)
      for k in range(bit_precision):
          file.write(W_bin[k])
    file.write('\n')
  file.close()

```

```
out_int.shape #out_ch, o_nij
```

```
torch.Size([64, 1024])
```

Soft shrink is applied to the output feature map to test soft shrink functionailty in Part3's reconfigurable output stationary design.

```
o_nij = 8
o_chan = 8
bit_precision = 16

fname = "out.txt"
file = open(fname, 'w')
out_int = int_softshrink(out_int, delta_out, lambda_real=0.5)
for i in range(o_nij):
  for o in range(o_chan):
    #out_int[7-o,7-i]
    if (7-o == 1 and 7-i == 0): print(out_int[7-o,7-i].item())
    if (out_int[7-o,7-i].item() >= 0):
        Out_bin = '{0:016b}'.format(round(out_int[7-o,7-i].item()))
    else:
        Out_bin = '{0:016b}'.format(round(out_int[7-o,7-i].item())+65536)
    for k in range(bit_precision):
        file.write(Out_bin[k])
  file.write('\n')
file.close()
```

```
14.0
/tmp/ipython-input-2470372827.py:11: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.det
  lambda_int = torch.round(torch.tensor(lambda_real / delta_out)).to(y_int.dtype)
```

```
# (X[:,0:3,1:4]*W[1,:,:].reshape(3,3,3)).sum() #equivalent  to out_ch = 1 and o_ij = 0
```

```
# torch.set_printoptions(profile="full")
# print(w_check[18:])
# print(W[2,2,:].reshape(3,3)) #o_ch = 2, in_ch = 2, 0:9
# weight_int[2,2,:,:]
```